

CMP-142 Object Oriented Programming

Lab 05

Topics:

- Overloading Stream and Relational Operators
- Aggregation and Composition
- Friend functions

Instructions!

1. This is a **graded lab**, you are strictly **NOT** allowed to discuss your solutions with your fellow colleagues, even not allowed asking how is he/she is doing, it may result in negative marking. You can **ONLY** discuss with your TAs or instructor.
2. Indent your code properly.
3. Use meaningful variable and function names. **Use the camelCase notation.**
4. Save your work frequently. Make a habit of pressing CTRL+S after every line of code you write.
5. **Do NOT use any global or static variable(s).** However, global named constants may be used.
6. Make sure that there are **NO dangling pointers or memory leaks** in your programs.
7. **Make sure that all functions have been implemented and can be called in the main function. Otherwise, no marks will be awarded for that function.**

In this lab, you will create a simple arcade game simulation that will incorporate various Object-Oriented Programming (OOP) concepts. You will implement classes representing players and games, and you'll explore fundamental OOP features like operator overloading, constructors, destructors, aggregation, composition, and friend functions.

Your task is to implement an arcade game simulation that revolves around a Player class, along with supporting classes like Stats and Game. Each of these classes will incorporate different OOP concepts.

[Task 01] Implement the Player Class

The Player class is the core class of the arcade game. It will represent individual players and provide attributes like health, score, and level. You will also implement methods to make players fight or train.

Create the **Player** class with attributes:

- **playerID**: Unique identifier for each player.
- **name**: Name of the player.
- **score**: Player's score (initialized to 0).
- **health**: Player's health (initialized to 100).
- **level**: The level of the player (starts at 1).
- **Stats**: An object to keep track of the player's statistics (covered in Task 4).

CMP-142 Object Oriented Programming

Lab 05

You should create a **constructor** that accepts parameters to set **playerID**, name, and optional parameters for health and score. Implement a **destructor** that displays a message when a Player object is deleted.

fight(): This function simulates a fight between two players. The player with the higher score will win, increasing their score, while the loser will lose health points.

train(): A function to increase the player's score and level to simulate training.

[Task 02] Overload Relational Operators

- Overload the **<** operator to compare Player objects based on their **scores**. This will allow you to directly compare if one player has a lower **score** than another.
- Overload the **==** operator to compare players by their **playerID**. This can be useful for determining if two players are actually the same.

[Task 03] Overload Stream Operators

- To facilitate easy input and output of player data, you will overload the **stream insertion (<<)** and **extraction (>>)** operators.

[Task 04] Create and Integrate Stats Class (Composition)

You will now create a **Stats** class that will be used within the **Player** class, demonstrating the concept of **composition** (a "has-a" relationship). Create a **Stats** class to keep track of player statistics, such as **wins**, **losses**, and **matchesPlayed**.

Also Implement the following functions:

```
void recordWin();
void recordLoss();
void displayStats() const;
```

Add Stats to Player Class (Composition): Add a Stats object as an attribute to Player, indicating that each player has a set of statistics.

```
class Player {
private:
    Stats playerStats; // Composition: Player "has-a" Stats
    // Other attributes here...
public:
    // Constructor, destructor, and other functions...
};
```

[Task 05] Create a Game Class (Aggregation)

Next, you will create a Game class that will **aggregate** multiple Player objects to simulate a game. Aggregation indicates a "whole-part" relationship.

CMP-142 Object Oriented Programming

Lab 05

Define the Game Class: Create a Game class that has a collection of Player objects.

```
class Game {  
private:  
    int gameID;  
    Player** players; // Pointer to array of Player objects  
    int playerCount;  
    int capacity;  
  
public:  
    Game(int id, int maxPlayers);  
    ~Game();  
    void addPlayer(Player* player);  
    void startGame();  
};
```

You will add players to the Game class using the dynamic pointer array. The destructor will ensure that all memory allocated on the heap is properly cleaned up.

[Task 6] Use Friend Functions

Create a friend function that can compare two Player objects and determine if they are compatible (i.e. their absolute difference is less than 20).

```
friend bool arePlayersCompatible(const Player& p1, const Player& p2) {  
    // Players are compatible if their scores are close  
}
```